

Setup:

- * gwen 2.5 - 0.5 B as base model
- * uses grouped Query attention or GQA
so 14 Query heads & 2 KV heads
- * ran into dependency issues,
(order based project & transformers)

Attention Sinks Visualise:

- * Layer 0-2 : 2-5% of mass on the first few tokens
- * Layer 3-21 : 30-50% of all mass binds on the first token
- * Layers 22-23 : collapse back to 10%
{ entirely posn 0 & not 4 tokens }
{ , others just barely register }

Why the layer thing happens

1) it does basic token mixing & figure out & hasn't built represent-

2) G-213 model does semantic processing

& when a head doesn't find what it looks for, just dumps in sink

3) Drops because the job shifts from understand context → predict tokens.

so more local, output facing work

so sink effect fades

Eviction policies

1) sliding window : $\rightarrow$ strawman

drop everything other than last N tokens, $\Rightarrow$ drops $\text{sink} \leftarrow$ tokens

2) Sink aware :

permanently pin first few tokens
& slide a window over rest

3) H2O :

track cumulative attention
across all generation steps

& keep the top, evict the rest

H2O bias :

Like new tokens can't do anything
others have had a lot of time (40 vs 2)

so by construction it keeps & protects
old tokens $\Rightarrow$ contaminated by age

Position Encodings after Eviction:

Rope gets baked into cached @ their absolute positions, so even after evicting old, they carry old rotary phase so when next token arrives, its query gets RoPE'd @ a posn. compared cached length anymore.

Fix:

override cache_posn so new query gets placed @ posn : current (cache size, not true index
-) matches cache length it attends over.

Stricter fix would be to encoding cached keys, with new relative posn.
(time issue)

Evaluation:

* Test 1:

* single text repeated 40 times as eval text,

Outputs: * 1-16 → baseline perplexity

* H2O < slide < sink
(3) (29) (36)

* no eviction baseline
→ recovered needle
while other policies failed

slide → looped
sink → partial
H2O → ramble

* Test 2:

used JJK wiki to test

27608 chars pulled
(using user-Agent)

Analysis:

@ 7406 tokens:

- every single policy missed
- issue is model too small

@ 2349 tokens (200 words)

- no eviction baseline recovered

↓ ON Real Prose, H2O won

H2O	>	slide	>	sink
(2200)		(100)		(232)

Why?:

in repeated text, accumulated eth.
concentrate on right one by pure chance
but on varied prose, the trick fails
+ sink's direct approach wins

Quality vs Memory:

Swept all policies across cache budgets
32, 64, 128, 256, 512 tokens on same
article.

Baseline perplexity: 11.94

* Analysis:

- * sink wins @ every single budgets

- * H2O starts as the worst, but improves fastest, as it needs enough budget to retain useful stuff. @ higher budgets, it throws away sink tokens

* Needle recovery:

@ 128 → all fail
@ 256 → Sink recovers

@ 512 : slide 2 H2O

∴ Sink aware best.



But when:

- Multi turn chat agent: H_2O aware
complexity $\leftarrow$ (model stability) with not clear benefit

- Long document summariser: H_2O
↳ as important info scattered all through,